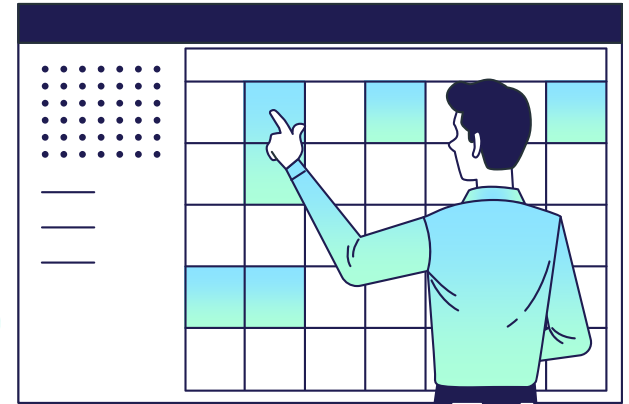




**Zaštita
podataka**

Razni zadaci (Python)



ZADATAK 1 – RSA

- Ana je šifrovala poruku napisanu na engleskom jeziku RSA algoritmom svojim javnim RSA ključem ($e = 509$, $n = 64507$). Poruka je šifrovana tako što su grupisana dva po dva slova prikazana u ASCII formatu i pretvorena u brojeve na sledeći način: $XY \rightarrow \text{ord}(X) * 256 + \text{ord}(Y)$, gde 'ord' funkcija vraća ASCII vrednost karaktera, a potom je takva vrednost prikazana u heksadecimalnom formatu na širini od 4 cifre.
- (npr. „AB“ = $65 * 256 + 66 = 16706 = 4142h$)
- Odrediti Anin privatni ključ d , a potom dešifrovati poruku.

ZADATK 1 – RSA

Šifrovani tekst:

```
4209c15b6135828160d357324302b4b8b8b7563aa801d2b2c15b0cc1c15b082df1f2063eecbe618f0a72
da373abae48460d30a724e4ae484266b68cdf27fa339545434d127a89094e061035668cd266b5e5e6144
6562ccb060d31d0ae4e57df1618f384b6865266b68cd430255eef1f2063ee6f75656739057321ce90898
266bd7ea002073909a29604f2d403833790ce4846c1560f060d394145e5ef06f1d4161448f1b0cc161cb
6144ee55f0f13833495cf1f2063ed9840f10f9d41ce9b4b81ce19aefdf03503c9d30e1a8f6063fc9f606
ee5590946b055f5fc190c15bc0612916c15b0cc1c15b8b6ba212de92c2c1c15b4c1034d10cc1c15b1e1b
6969bf37e91f29161ce9ba18618f4e4a80c4ab121d4161442d4000acf33e5f62e48460d3d937caaaaf00
a9e57b1da0d461351ce160d35732389e1ce19093f1f6f6bdee556144a2126969bf37ecbe383324284ced
38c8a80174942773ee5560d3f72a08981ce90898d5a7caaa1b02637f2d40a2126144b9bcf958277360d3
e746a2120ba438c8f27f2e21b975389ea929a21269690d0cde381c8819f5a212de92b9bc1c88f0f1bffb
61442d40ab12f6062e215e5e573275602b2fcaaa75225f5fa1a8604fc2c1614465627390f9d456563801
266b68cd7df1c8e238330cc161cb43021a2d0b410d0cb7abf27f48e10f43f27f8f1b1e3e7b7cde924dba
73905f6231117b6bc320bf37bd50c15b0fb26350e4846144656273902b8bcb0b61442d40614457323635
614465627390b28df06fbf37d700b8b761445f6211b848e1f6063fc960d35732503c7e7e6c07e48460d3
d7eaae550f430356839f5732604f61cb0bbeb4b8e1c11cb1b614a0d4ba1811b81d0acfbcb0b410d0ca2a8
2d40bffb7390db1eaca8c15bcfbcb1ce180dc60d357322627e1c1c320
```

ZADATK 1 – Rešenje

```
from math import sqrt
n = 64507 #  $n = p \cdot q$ 
e = 509
p = None
q = None
for i in range(3, int(sqrt(n))+1, 2):
    if n % i == 0:
        p = i
        q = n // i
        break
phi_n = (p - 1) * (q - 1)
d = pow(e, -1, phi_n)
print(d) #  $d == 2389$ 
```

ZADATK 1 – Rešenje

```
ciphertext = "4209c15b...c320" # 4209h -> 16905, ...
block_length = 4
block_count = len(ciphertext) // block_length
cipher_blocks =
[int(ciphertext[i * block_length : (i+1) * block_length], base=16)
 for i in range(0, block_count)]

# cipher_blocks -> [16905, 49499, ..., 49952]
```

ZADATK 1 – Rešenje

```
plaintext_blocks = [pow(c, d, n) for c in cipher_blocks]
plaintext = ""
for i in range(len(plaintext_blocks)):
    plaintext += chr(plaintext_blocks[i] // 256)
    plaintext += chr(plaintext_blocks[i] % 256)
print(plaintext)
# plaintext =
She walks in beauty, like the night
    Of cloudless climes and starry skies;
...
A mind at peace with all below,
    A heart whose love is innocent
```

ZADATAK 2 – RSA + Vigenere

- Ana i Branko razmenjuju poruke *Vigenere* algoritmom sa *autokey* poboljšanjem uz korišćenje engleskog alfabeta.
- Ključ za *Vigenere* algoritam Branko šalje Ani enkriptovan RSA algoritmom čiji je javni parametar $n = 868165516507$, a Anin javni ključ je $e = 52127$. Prilikom razmene ključeva, Ana je primila sledeću poruku koja sadrži enkriptovan ključ: 453706198292. Kada se dešifruje ova vrednost, dobiće se 11-cifreni broj koji treba pretvoriti u 7 slova za *Vigenere* ključ po šemi `ab c de fg h ij k`, gde su `a, b, c, d, e, f, g, h, i, j i k` cifre ovog 11-cifrenog broja.
- Dešifrovati sledeću poruku koju ja Branko poslao Ani:
- SOYXBAEKOTVLHTLBYZUKTSBYLCXVQACUIIRLEZVGAOKQKRXPXOBQYCRAR

ZADATK 2 – Rešenje

```
from math import sqrt
n = 868165516507
e = 52127
p = None
q = None
for i in range(3, int(sqrt(n))+1, 2):
    if n % i == 0:
        p = i
        q = n // i
        break
phi_n = (p - 1) * (q - 1)
d = pow(e, -1, phi_n)
print(d) # d == 634480452063
```

ZADATK 2 – Rešenje

```
encrypted_key = 453706198292
key_number = pow(encrypted_key, d, n)
key_digits = [key_number // pow(10, i) % 10 for i in range(10, -1, -1)]
key = [key_digits[0] * 10 + key_digits[1],
       key_digits[2],
       key_digits[3] * 10 + key_digits[4],
       key_digits[5] * 10 + key_digits[6],
       key_digits[7],
       key_digits[8] * 10 + key_digits[9],
       key_digits[10]]
# key == [25, 0, 18, 19, 8, 19, 0]
```

ZADATAK 2 – Rešenje

```
def encode_letter(c):  
    return ord(c) - ord('A')
```

```
def decode_letter(c):  
    return chr(ord('A') + c)
```

```
def decrypt_letter(c, k):  
    return (c - k) % 26
```

ZADATK 2 – Rešenje

```
ciphertext = "SOYXBAEKOTVLHTLBYZUKTSBYLCXVQACUIIRLEZVGAOKQKRPXOBQYCRAR"
plaintext = ""
for i in range(0, len(ciphertext)):
    decrypted_value = decrypt_letter(encode_letter(ciphertext[i]), key[i])
    plaintext += decode_letter(decrypted_value)
    key.append(decrypted_value)
print(plaintext)
# plaintext = TOGETHERANRSAPUBLICKEYANDANRSAPRIVATEKEYFORMANRSAKEYPAIR
# (TOGETHER AN RSA PUBLIC KEY AND AN RSA PRIVATE KEY FORM AN RSA KEYPAIR)
```



ZADATAK 3 – Feistel algoritam

- Ana je poslala Branku poruku šifrovanu algoritmom baziranom na Feistel strukturi sa 16 rundi koji ne koristi ključ, već je sama funkcija f tajni podatak koji znaju samo Ana i Branko, a koja je definisana sa $f(x) = 7^x \bmod 26$. Poruka se sastoji samo od slova engleske abecede, pri čemu je prilikom računa slovo A kodirano sa 0, slovo B sa 1, itd.
- Dešifrovati poruku koju je u toku prenosa presreo Cvetko, koji poznaje korišćeni algoritam, imajući u vidu da je svako slovo predstavljeno na širini od 5 bita.

ZADATK 3 – Feistel algoritam

Šifrovani tekst:

```
1011110000010001101001111000101000001111001101110011
0110010111011001011000100010110101000001101101011101
1001010000100010011110001001000000100001111000110001
11000111110100001100000000011001011000001111
```

ZADATK 3 – Rešenje

```
def feistel_function(value):  
    return pow(7, value, 26)  
  
def feistel_round(left, right):  
    return right, left ^ feistel_function(right)  
  
def feistel_algorithm(left, right):  
    for _ in range(16):  
        left, right = feistel_round(left, right)  
    return right, left
```

ZADATK 3 – Rešenje

```
def decode_letter(c):  
    return chr(ord('A') + c)  
  
def decrypt_message(ciphertext):  
    plaintext = ""  
    for i in range(0, len(ciphertext) // 10):  
        x = int(ciphertext[i*10:i*10+5], 2)  
        y = int(ciphertext[i*10+5:i*10+10], 2)  
        x, y = feistel_algorithm(x, y)  
        plaintext += decode_letter(x)  
        plaintext += decode_letter(y)  
    return plaintext
```


ZADATAK 3 – Rešenje

```
ciphertext = "10111...01111"
plaintext = decrypt_message(ciphertext)
print(plaintext)
# plaintext = ITISONLYWITHTHEHEARTTHATONECANSEERIGHTLY
# (IT IS ONLY WITH THE HEART THAT ONE CAN SEE RIGHTLY)
```

ZADATAK 4 – CBC mod funkcionisanja

- Nikola je poslao Zorani sliku veličine 27x40 šifrovanu *Vigenere* algoritmom sa *autokey* poboljšanjem i CBC modom funkcionisanja. *Vigenere* algoritam je modifikovan da vrši obradu na nivou heksadecimalnih cifara, pri čemu algoritam radi sa blokovima veličine jedne RGB vrednosti piksela (6 heksadecimalnih cifara).
- Nikola je razbio sliku na blokove tako što je iz slike izvukao niz RGB vrednosti (u heksadecimalnom formatu u opsegu od 000000h do FFFFFFFh) svih piksela koji čine sliku, a potom je izvršio šifrovanje.
- Ključ za *Vigenere* algoritam je **2A8h**, a inicijalna vrednost CBC moda funkcionisanja je **A58B91h**.

ZADATAK 4 – CBC mod funkcionalisanja

• Dešifrovati primljenu sliku čiji su pikseli opisani sledećim stringom:



"646e4c626e485e6e3d5d713c5d70425f7e7b6d455769415c6c485d6485d6416273476072464e764b6435b6e40596b3b596b395667335633255643956663f586746566745566b42556b3d56693c757f5a6e794f6a7b647774a36
87c4b677a4c63744865744969794c6c794b6e7b4f6a764e6a79506e7e5170804f6e7f4b6e7f4b66774b6d7b4a6e7b50667546e775526e7e597081576d80526c7d517d865975814f778547a894e77885474845371824e6e7c47758
3487782487174a57a80507f8559858b5f777f4c717c4478824d7986507b855374825177825879845c7e89617b885d7b885a7a885776855a18a5b7f89557d884e838e547a87527e8a58808d57808517e74884884b8a89519085a827b4
f7817478784598b8c60848a5a878f5c7c86527985537580557b8459848b5f828a59838b58818b577e8a5c838c5f87915f89905c8a915b858c59888f5c8e97608c945b8d9156989359978a535544104d380b3e2a072412003124137673549
0926a7f875487f895487915f8991609046287925d8f925b8c935d8a9261838c5f8e9665969a68947f6296996491966009b9c94c642955e9b9863a3a6d574203f8c8041977a40684b1f412412250f040f03008b8a6c939b6c87925a87905b939
a69497609995f9494589255c8e9762899162969e6b9599669e9e689f6a09f6a961669e968b1a9763521b0d97b9f55f6c77aeac574c6a05f69481f2f1400200f08211b0da2a77f919b679ca671989f699e9f65a3a1649a995999a5
e969d98686e59ba276959766af79abab6fb0b2739fa368a4a26faa27c432d00cfaf62d8b358e2b86c249a529d75444d2d14210d0c080000777c5e7b8759889464949d68a4a56ba8a767a1a060a2a2669ea4745e64a666b4b774774c8
88b5685864c8a8b4fad077bab8888d7e61422803b964fcdca34fb88742af814692673a59371e1f0b0d180f143c3f2c636e4c6e7a54758153a4a971aeb071a9a96b9091587a81583e432f62664d70724d6b6c40602306c6e3c8a8c5d989
46f75645050331595702cba903e9564219c6b319f75456b492e220d12190f1833372946503758654756623ca0a8779a9e76a3a36d7f8351866b6b5255425a5e47606044595a385658334e4f2453537564f3c55423b442812e3c07eddb56
0bb8d42efc27fc198605837162410121c121d33352a616a55606c54546143848c67959c7292946c8286617e836c4c4d3b3e402b4f5139575a3f5c5f444b4d354e4d394a42353d29282c0f00c2a063b79041845a0ed0a765926a373c1d012
b161b190f1a24241c363d2b3e4a3448543c616a4d616849585d3d5c6045595e4a3c3b274241d253553f50543d5c82384b4f384d4f3a4b43363d2a24321706937141bf9c5a88631fab844b8c65443c1c12c1720170b152320194b4f3e454
e393d493348513c5b634c535841484d375d604d3c3022474435494a3844a4f393f4730484d37494a385d564404c3a303a1e10381700bb996bcb98628662325c381e36180e2b141c1d1019241f194e5042585f4d4a533e4c5542444d3805804
14f543e585b48362d28413d44041314f544048503b4b533e545543756e5e57453734190e300e053b160664401e42ba1045240545281626101222131b1f51348463a494b3d4b533e424c343b452c4a5338595d445657453c2e2b33432d4
0413150333e5b5e4969685475705d766d5e635346331d123819143915058f6c4678586266d4bd7e603e3a1f141d090a261817423b335d5c4a585c435c61415f65435f6444575a3f66644f5133c3663554a62604b67674d554e3454452e4d3
e2b534353b2e282b1a10341c0262461f7a8e4bb79347ba944bb591755b1f43251b3422201d130a3731a45431474789659669465d593c5e5a16044385e45334e4228635838675433544d73d72174731242f1e172c1908775c2
fd7b785deba6dbb4574b85dd1aa5f73491f5f3b25381e1128160a3e321a7f71519a8f627f71446661396c623f65a3d684513166c43366c2c29876e458065365d411963482d4c2f1d2b0e004ad2c0bda766f0c7759e9b60edc3616eb65cd
3a9557b5019532903341100583c242f170b493628826e5594805d9882598c794f685536494326f5f16a84618917165492f5e422c111023f1b05b786ed5edc07cf3c56ff5c9b6ae6f0eac3669da657f6471366401b9a75589e
85553936250c0f361f7634a3678b534967a55967e5c67422f6d452c7e502d794f257e5f538d72543017133315154a240f9c9c72f9c78e8cbb6cf4d7bfd87f6d6586c6a8687c603972564147261f2c0e10240d172511c241019412825
b5532a7b5644947559543024835a488d53c8456327f5c3c361b08271143d242866423269411d7d541eac8542c5a35bcfb16bb69b646a542b36240c27150b3a1d1f361b221706101e10d19282b35236744a643d386746354321225
52f26835248b5d38b5f49391d123f292c32d292b533b3142291314b266e592c614f1f59471f58460261f121b11083528204a362f402b2632625291e2261b2929172552343e603d416345434a2a2f532e261a281d11d1f12
e1a2a171d3a292f3422202e3d2493c2950462b4731c5e246302a1c191e1216292023291f1d38271f604e44493e3a271e12139431212e462c355a3b41573a3f5639338a6a5d99755ba18168523c313421232c1e2a36263336252d2d1f1
f3d32e32281c4f4303d301f2c1e1d22171f1e141f2c212944535253413d1e0f121c11171f131d3d2d3741282e7f15458604347f5443b5d4133654a4375841f2f2d1b174132373826322f1d2b2a1a27312c31262a382d2b3f32293b2c251
11132216201d152228202b33262d1d0e1125181f1d10192c1f293222c321d2651383e53383d5a13d563394a362d36251d3a2c2b47383b4431333522829197272918282a1b204133333f2322a191f24172018111916131e25212f120
91a1e121290d191d11b190d1902231311e314423d353a4057413432f303622140c2b392543823207f1f251113d32e332b1b26211322251b261b1521191621151011081d24182c261a281d11d1f142
2e21332e1f34402f41550447514142301d1f412a3044272956332d5f3128421d15321310200a0c1d0c12261a1e3c1262a2419271f1627151120171322140d1d130a1b170a1b2012231b0d1c1c0f201a1221e1729261d303429394a434b4
b3c41e1d232c172056373d774e486a3e316f4839583522826651f549342c15071b07001d0e11e141f1915260e0b1e50e2104b1c1c0e1f2002f21231427170b1f170f241e1b2elf1b2c221c2a4137423d303931242e2a1826563c495c3
83a5b352a60c24ba9771d2ad78d3ac71ba8f5c966c442f13001b09071d14250f0b220f0a1e120b1d120519e0f2624162f1a102819162b201f31221f32720323633433c323d2b1f2d2011242f1a2d4125314c2a284e2b17896239a4778
7ba8a0c99450c6935abd966f7d5ea16000317071214050a2e1d3f192625102118061ca0e2429243alf1c311d822b2624363427393c34411911201c102615051f2512283117222f0d04592d108b562aa37132b0804b04834893632a
1714b865a41845b47774e3a5b34236d48404f2f3030161f1706162c20341e162b1e62b251c2f36293a221b2b1d427190d2524173117092213000d62413c976d55b98a6080511d8f642da075409b68398522a6431146a38215721706b3
e2b4b21393f1b13871f2511000210361a1910251c1328261b2c3b43c221833160d2520142a1f13274182e1a901364483661646648861b635663f780482575b3e6e3f2b72463d42191704173513113619130219321a01a17081
b0f0319170b12101321f241c2d3e2c381b112c140a23201428160b1c14091a2717214429227d5487485834921074c291548625164c261b4921e3c181c2203b032309121d0811a081617061815081e0f03b01f31e10271f10232514243
512d2d1152b1d1251a0f20d12221e1323231320250c104f31313614123d1b1936181a3b1e233b1e23280d142912130f01f1509170f0619003010c04190d051e300012160a221b0e221214241222301b2a1c1026180c20170
c1d1e132315071626101d2f1522d04e1d2506152f1425260f1f1f0a1b50130e0001207150d091a0603160a071c070419083190b031a0900151b0e2119081b1a071a271225190c281407211120140a221108191f122310f1d201
61a011621081e210b20130215190a2112061c0a03150602110906170504160502150603160702160a021707001313061a150619170619210f251a0d23130624130826130922160d201b10201f0e211d0a1e1e602022b525202f150a0b221
306200a021a0c091c0b0a1a0e0d1f0807190906190603160b061a0b03180b02151004181405181304171b0c23"

ZADATAK 4 – CBC mod funkcionisanja

- Nakon što se dobije niz tekstualnih blokova koji predstavljaju piksele originalne slike, iskoristiti sledeći kod da se od njega generiše slika:

```
def save_pixels(pixels, filename):  
    col = []  
    row = []  
    for i in range(len(pixels)):  
        pixel = pixels[i]  
        row.append([int(pixel[0:2], 16), int(pixel[2:4], 16), int(pixel[4:6], 16)])  
        if i % 27 == 26:  
            col.append(row)  
            row = []  
    img.imsave(filename, np.uint8(col))
```

ZADATK 4 – Rešenje

```
def decrypt_one_symbol(c, k):  
    return '{:x}'.format((int(c, 16) - int(k, 16)) % 16)  
  
def vigenere_decrypt(ciphertext, key):  
    plaintext = ""  
    for i in range(len(ciphertext)):  
        plaintext += decrypt_one_symbol(ciphertext[i], key[i])  
        key += plaintext[i]  
    return plaintext
```

ZADATK 4 – Rešenje

```
def string_xor(a, b):  
    return '{:06x}'.format(int(a, 16) ^ int(b, 16))  
  
def cbc_mode(ciphertext_blocks, key, iv):  
    plaintext_blocks = []  
    for ciphertext_block in ciphertext_blocks:  
        plaintext_block =  
            string_xor(vigenere_decrypt(ciphertext_block, key), iv)  
        plaintext_blocks.append(plaintext_block)  
        iv = ciphertext_block  
    return plaintext_blocks
```

ZADATK 4 – Rešenje

```
ciphertext = 'eb61...ee93'  
key = '2a8'  
iv = 'a58b91'  
ciphertext_blocks =  
[ciphertext[6*i:6*(i+1)] for i in range(0, len(ciphertext) // 6)]  
plaintext_blocks = cbc_mode(ciphertext_blocks, key, iv)  
save_pixels(plaintext_blocks, 'decrypted.jpg')
```



ZADATAK 5 – Diffie-Hellman razmena

- Marko je poslao Neveni sledeću šifrovanu poruku:

UQFCECBWDGELOLSAQAJXOUUHOMBTEHFBXCTKFEFNPWFMUQSLQUBEFKFB

- Poruka je šifrovana algoritmom koji radi tako što svako slovo teksta preslikava u slovo koje se nalazi n mesta kasnije u alfabetu (uz kružno vraćanje na početak alfabeta u slučaju prekoračenja), gde je n cifra na odgovarajućoj poziciji u ključu (za prvo slovo – prva cifra, za drugo slovo – druga cifra, ... , pri čemu se za slovo na poziciji $n+1$ ponovo vraćamo na prvu cifru, itd.).
- Ključ za šifrovanje su razmenili Diffie-Hellman algoritmom sa parametrima: $\alpha=19$, $q=200003$. Ako se zna da je Markov privatni ključ $Xm=103$, a Nevenin $Xn=148134$, odrediti ključ koji je korišćen za šifrovanje, a potom dešifrovati poruku.

ZADATAK 5 – Rešenje

```
q = 200003
```

```
alfa = 19
```

```
xm = 103
```

```
xn = 148134
```

```
ym = pow(alfa, xm, q)
```

```
key = pow(ym, xn, q)
```

```
print(key)
```

```
# 191217
```

ZADATK 5 – Rešenje

```
key_digits = []  
while key != 0:  
    key_digits.append(key % 10)  
    key = key // 10  
key_digits.reverse()
```

ZADATK 5 – Rešenja

```
def decrypt(l, k):  
    return chr((ord(l) - ord('A') - k) % 26 + ord('A'))  
  
ciphertext = "UQFC...FKFB"  
plaintext = ""  
klen = len(key_digits)  
for i in range(0, len(ciphertext)):  
    plaintext += decrypt(ciphertext[i], key_digits[i % klen])  
print(plaintext)  
  
# THE ADVANCED ENCRYPTION STANDARD AES WAS DEVELOPED TO REPLACE DES
```

ZADATAK 6 – ElGamal digitalin potpis

- Ana je poslala Branku poruku koju je digitalno potpisala El Gamalovim digitalnim potpisom i digitalni potpis ($S1 = 8121408200$, $S2 = 24148703915$). Ako se zna da su javni elementi digitalnog potpisa: $q=1000000000003$ i njegov primitivni koren $\alpha=1000133$, da je Anin javni ključ $Ya=60620549947$ i da se za izračunavanje hash-a koristi funkcija:

`binascii.crc32(bytes(text, 'utf-8'))`

- Odrediti da li Branko može da bude siguran da su poruka i digitalni potpis oni koje je poslala Ana.

ZADATAK 6 – ElGamal digitalin potpis

- Poruka:

```
text="""This note describes the eXtended Merkle Signature Scheme (XMSS), a
hash-based digital signature system that is based on existing
descriptions in scientific literature. This note specifies
Winternitz One-Time Signature Plus (WOTS+), a one-time signature
scheme; XMSS, a single-tree scheme; and XMSSMT, a multi-tree variant
of XMSS. Both XMSS and XMSSMT use WOTS+ as a main building block.
XMSS provides cryptographic digital signatures without relying on the
conjectured hardness of mathematical problems. Instead, it is proven
that it only relies on the properties of cryptographic hash
functions. XMSS provides strong security guarantees and is even
secure when the collision resistance of the underlying hash function
is broken. It is suitable for compact implementations, is relatively
simple to implement, and naturally resists side-channel attacks.
Unlike most other signature systems, hash-based signatures can so far
withstand known attacks using quantum computers."""
```

ZADATAK 6 – Rešenje

```
import binascii
text = """This note...computers."""
m = binascii.crc32(bytes(text, 'utf-8'))
q = 1000000000003
a = 1000133
S1 = 8121408200
S2 = 24148703915
Ya = 60620549947
v1 = pow(a, m, q)
v2 = pow(Ya, S1, q) * pow(S1, S2, q) % q
# v1 = 8087557471, v2 = 22137472393 --> Branko nije siguran
```

ZADATAK 7 – PGP

- Na predmetu Zaštita podataka, asistenti i studenti razmenjuju elektronsku poštu korišćenjem PGP šeme koja koristi sledeće algoritme prilagođene za rad sa jednim bajtom podataka (za neki ulaz X , izlaz Y , simetrični ključ K i asimetrične ključeve PU i PR veličine 1B):

```
Y = X xor K           # simetrični algoritam
```

```
X = Y xor K
```

```
Y = (X + PU) mod 256  # asimetrični algoritam
```

```
X = (Y + PR) mod 256
```

```
Y = X * X mod 256     # heš funkcija
```

ZADATAK 7 – PGP

- Kako poruka koja se šalje može da bude proizvoljne veličine, algoritmi se koriste na sledeći način:
- # simetrični i asimetrični algoritam
 - svaki karakter (bajt) poruke se zasebno obrađuje, a potom obrađeni karakteri konkatenuiraju
- # heš funkcija
 - ASCII/UNICODE vrednosti svih karaktera (bajtova) poruke se sabiraju i moduluju sa 256 kako bi se dobio jedinstven ulaz veličine 1B.

ZADATAK 7 – PGP

- Asistenti imaju sledeći par ključeva: **(PU_a, PR_a) = (123, 133)**
- Studenti imaju sledeći par ključeva: **(PU_s, PR_s) = (201, 55)**
- Asistenti su studentima poslali poruku uz korišćenje servisa autentikacije i tajnosti, pri čemu su oba servisa zasebno i nezavisno iskorišćena (nasuprot originalnoj šemi gde se koriste jedan za drugim). Studenti su primili enkriptovanu poruku (poruka je kodirana tako što je svako slovo poruke predstavljeno svojom ASCII/UNICODE vrednošću na širini od dve heksadecimalne cifre), enkriptovan ključ sesije i enkriptovan potpis poruke:
 - `enc_message` =
"59677162756c2f090956707366706d6c23707766237166706a6f6a236c7562692379626762776268220909536c79677162752f09706223737166676e667762"
 - `enc_session_key` = 204
 - `enc_signature` = 9

ZADATAK 7 – PGP

- Dešifrovati i verifikovati primljenu poruku i odgovoriti na sledeća pitanja:
 - Koja je vrednost simetričnog ključa?
 - Koju poruku su asistenti poslali studentima?
 - Da li studenti mogu da budu sigurni da su im asistenti poslali ovu poruku? Ukratko objasniti.

ZADATAK 7 – Rešenje

```
def sym(message, key):  
    return message ^ key;
```

```
def asym(message, key):  
    return (message + key) % 256
```

ZADATK 7 – Rešenje

```
enc_message =  
"59677162756c2f090956707366706d6c23707766237166706a6f6a236c756269237962676277626822090953  
6c79677162752f09706223737166676e667762"  
  
enc_session_key = 204  
enc_signature = 9  
  
PUa, PRa = 123, 133  
PUs, PRs = 201, 55  
  
session_key = asym(enc_session_key, PRs)  
signature = asym(enc_signature, PUa)  
print("Session key: ", session_key) # Session key: 3  
print("Signature: ", signature)      # Signature: 132
```

ZADATAK 7 – Rešenje

```
message = ""
hash = 0
for i in range(0, len(enc_message), 2):
    input = int(enc_message[i:i+2], base=16)
    output = sym(input, session_key)
    message += chr(output)
    hash += output

print(message)

# Zdravo, ... Pozdrav, sa predmeta
```

ZADATAK 7 – Rešenje

```
hash = (hash * hash) % 256;  
print("Message hash: ", hash) # Message hash: 132  
print("Hashes are the same: ", hash == signature)  
# Hashes are the same: true
```

PITANJA?

rti.etf.bg.ac.rs/rti/ir4zp

CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik** and illustrations by **Stories**

